

MOA 실시간 음성 대화 설계안

대상: 고령자 음성 챗봇 · 작성일: 2026-06-20 · 목적: 윤현 브랜치의 WebSocket/VAD 초안을 실제 서비스 흐름으로 발전시키기 위한 기준

핵심 원칙. 실시간성보다 “끊기지 않는 대화, 큰 실패 없는 UX, 개인정보 최소 처리”를 우선한다. 음성 원본은 메모리에서만 처리하고, 고령자 UI에는 의료 수치·질병 판단을 보여주지 않는다.

1. 전체 흐름

앱 —(HTTPS 로그인)—> Supabase / MOA API
앱 —(WSS + access token)—> /chat/ws

1. start_session
2. audio_start
3. PCM 16 kHz / mono / signed 16-bit 청크 전송
4. 서버 VAD → 발화 확정
5. STT → 대화 모델 → 문장 청크 응답
6. 서버 TTS 또는 클라이언트 TTS 큐 → 순서 재생
7. reply_end → 다음 듣기 상태

2. 실시간 음성 전송 규격

항목	권장 설계	이유
연결	wss://api.../chat/ws, 세션당 하나의 WebSocket	상태·오디오·응답 청크를 한 연결에서 순서 보장
오디오	16 kHz, mono, PCM signed 16-bit little-endian, 20~100 ms 청크	STT에 충분하고 대역폭·지연을 줄임. 100 ms라면 청크당 약 3.2 KB
클라이언트 → 서버 제어	start_session, audio_start, audio_end, cancel_turn, ping	사용자가 말하기를 취소하거나 재연결해도 서버 상태를 명확히 정리
서버 → 클라이언트 상태	listening, speech_detected, thinking, reply_start, reply_chunk, reply_end, error	캐릭터 표정, 마이크 UI, TTS 큐가 같은 상태를 공유
순서	모든 메시지에 session_id, turn_id, sequence 포함	늦게 도착한 이전 응답을 버리고 재연결 후 중복 재생을 방지

3. 권한과 개인정보

- WebSocket 연결 시 Authorization: Bearer <access token> 또는 단명 서명 티켓을 사용한다. URL 쿼리 토큰은 로그·프록시에 남기기 쉬우므로 피한다.
- 서버는 연결 직후 토큰을 검증하고, 고령자 계정만 음성 대화 세션을 만들 수 있게 한다. 보호자는 본인용 테스트 세션을 명시적으로 분리한다.
- 세션마다 서버가 session_id를 발급하고, 기존 세션 소유자만 재개할 수 있게 검증한다.

- PCM 버퍼·WAV 임시 파일·STT 입력은 처리 후 즉시 폐기한다. 로그에는 발화 원문, 음성 URI, 토큰을 남기지 않는다.
- 최대 발화 시간(예: 60초), 최대 버퍼 크기, 분당 요청 수를 제한해 비용과 악용을 막는다.

4. VAD와 턴 확정

단계	동작	사용자 경험
LISTENING	낮은 비용의 에너지/VAD로 발화 시작 감지	마이크가 켜진 큰 버튼과 “듣고 있어요” 표시
SPEECH	음성 청크를 버퍼링, 최대 길이 초과 시 안전하게 종료	파형과 경과 시간 표시
PAUSE	무음 600~900 ms에서 부분 전사·문장 종결성을 함께 판단	말을 잠깐 멈춰도 바로 끊지 않음
COMMITTED	종결 어미 또는 최대 대기 시간에서 턴 확정	“생각하고 있어요”로 전환

주의: 윤희 브랜치의 Hybrid VAD는 좋은 초안이지만, 실제 PCM 전송이 연결된 뒤에만 도입한다. 먼저 버튼으로 녹음 시작·종료하는 안정적인 경로를 완성하고 자동 종료는 점진적으로 켜다.

5. 재연결과 실패 처리

상황	처리
일시적 연결 끊김	1초, 2초, 4초, 8초 간격의 지수 백오프로 최대 4회 재시도. 화면에는 “연결을 다시 확인하고 있어요”만 표시한다.
재연결 성공	가장 최근의 <code>session_id</code> ·마지막 완료 <code>turn_id</code> 로 <code>resume_session</code> 요청. 아직 확정되지 않은 오디오 버퍼는 재전송하지 않고 사용자가 다시 말하도록 안내한다.
응답 중 끊김	재생 중인 TTS 큐를 즉시 비우고, 서버의 마지막 확정 응답만 다시 요청한다. 이전 문장을 처음부터 중복 재생하지 않는다.
STT/LLM/TTS 실패	각 단계 오류를 분리한다. STT 실패면 “잘 듣지 못했어요. 천천히 한 번 더 말씀해 주세요.”, TTS 실패면 텍스트는 유지한다.
토큰 만료	WebSocket을 닫고 HTTPS refresh 흐름으로 토큰을 갱신한 뒤 새 연결을 만든다.

6. 서버 부하와 비용 제어

- **연결 제한:** 사용자당 활성 WebSocket 1개, 기기당 동시 대화 1개만 허용한다.
- **버퍼 제한:** 최대 60초 또는 2 MB까지만 메모리 보관. 초과하면 안전하게 중단하고 재시도를 안내한다.
- **작업 분리:** WebSocket 프로세스는 상태 전송만 담당하고, STT·LLM·TTS는 제한된 작업 큐에서 처리한다. 긴 외부 API 호출이 연결 이벤트 루프를 막지 않게 한다.
- **취소 전파:** 사용자가 새 발화를 시작하거나 화면을 나가면 해당 `turn_id`의 STT·LLM·TTS 작업을 취소한다.
- **관측:** 원문 없이 연결 수, 평균 턴 시간, STT/LLM/TTS 오류율, 재연결 횟수만 집계한다.

7. TTS 재생 큐

1. 서버가 `reply_chunk`를 문장 경계로 보낸다.
2. 프론트는 각 문장을 즉시 `/speech/tts`로 요청하되, 동시 합성은 2개로 제한한다.
3. 완료된 오디오는 `turn_id + sequence` 순서로 큐에 넣는다.
4. 첫 문장이 준비되면 즉시 재생하고, 다음 문장은 백그라운드에서 미리 받아 둔다.
5. 재생 시작에는 캐릭터 `talking` 감정, 재생 종료에는 원래 감정을 적용한다. 새 턴·취소·화면 이탈 시 큐와 오디오 리소스를 즉시 해제한다.

이 방식은 전체 답변 TTS가 끝날 때까지 기다리지 않아도 되므로, 첫 음성이 빠르게 들리고 캐릭터 동작도 자연스럽게 맞출 수 있다.

8. 구현 순서

1. 현재 버튼형 녹음 → STT → REST 챗봇 → 서버 TTS 경로를 안정화한다.
2. TTS 재생 큐와 취소 처리, 캐릭터 발화 상태를 먼저 완성한다.
3. WebSocket으로 텍스트 스트리밍만 연결해 재연결·순서·중복 방지를 검증한다.
4. 마지막으로 PCM 스트리밍과 VAD 자동 턴 종료를 feature flag 뒤에서 도입한다.

이 문서는 구현 제안이다. 실제 적용 전에는 Android/iOS 오디오 형식, 네트워크 환경, OpenAI 비용·제한, Supabase 토큰 갱신 방식에 맞춘 통합 테스트가 필요하다.